

ডেটাবেজ কুয়েরি — কনসেপ্ট থেকে প্রজেক্ট পর্যন্ত

নিউজ প্ল্যাটফর্ম প্রজেক্টে যত ধরনের কুয়েরি লাগবে তার সম্পূর্ণ ধারণা

বেসিক SELECT থেকে Window Function, Partition Pruning, N+1 সমাধান পর্যন্ত — প্রতিটা কনসেপ্ট এই প্রজেক্টের আসল টেবিল দিয়ে উদাহরণসহ

লক্ষ্য: ডেটাবেজ কুয়েরির বেসিক এতটাই শক্ত করা যাতে প্রজেক্টের যেকোনো ফিচারের জন্য কুয়েরি নিজে ডিজাইন করতে পারো

এই নোট কীভাবে পড়বে

এই নোট দুই ভাগে বিভক্ত। প্রথম ভাগে (Part ১-১১) SQL কুয়েরির প্রতিটা বড় কনসেপ্ট আলাদা আলাদা করে ব্যাখ্যা করা হয়েছে — প্রতিটা কনসেপ্টের জন্য একটা "সহজ উদাহরণ" এবং একটা "এই প্রজেক্টে বাস্তব ব্যবহার" উদাহরণ দেওয়া আছে। দ্বিতীয় ভাগে (Part ১২) পুরো প্রজেক্টের সবচেয়ে গুরুত্বপূর্ণ **real-world** কুয়েরি প্যাটার্নগুলো একসাথে — যেমন পেওয়াল চেক, ট্রেন্ডিং লিস্ট, ফিড জেনারেশন — ফিচার অনুযায়ী সাজানো।

এই নোট পড়ার সময় নিজে হাতে-কলমে **MySQL**-এ (বা **sqlite** দিয়ে) প্রতিটা কুয়েরি রান করে দেখলে সবচেয়ে বেশি উপকার হবে। শুধু পড়ে মুখস্থ করলে ধারণা পোক্ত হবে না।

মডিউল ১

বেসিক কুয়েরি বিন্ডিং ব্লক

SELECT, WHERE, ORDER BY, LIMIT/OFFSET — সব কুয়েরির ভিত্তি

কুয়েরি এক্সিকিউশন অর্ডার — এটাই সবচেয়ে গুরুত্বপূর্ণ জিনিস

আমরা SQL লিখি এক অর্ডারে (SELECT প্রথমে), কিন্তু ডেটাবেজ এক্সিকিউট করে ভিন্ন অর্ডারে। এই পার্থক্য না বুঝলে অনেক কুয়েরি ভুল লিখবে (যেমন WHERE-এ alias ব্যবহার করার চেষ্টা করা)।

এক্সিকিউশন ধাপ	কাজ
১. FROM	কোন টেবিল(গুলো) থেকে ডেটা আসবে
২. JOIN	অন্য টেবিলের সাথে যুক্ত করা
৩. WHERE	সারি (row) ফিল্টার করা — এখানে SELECT-এর alias এখনো ব্যবহার করা যায় না
৪. GROUP BY	গ্রুপ তৈরি করা
৫. HAVING	গ্রুপ ফিল্টার করা
৬. SELECT	কোন কলাম দেখাবে তা ঠিক করা
৭. ORDER BY	ফলাফল সাজানো — এখানে SELECT-এর alias ব্যবহার করা যায়
৮. LIMIT/OFFSET	সংখ্যা সীমিত করা

মনে রাখার কৌশল: **WHERE** কাজ করে গ্রুপিং হওয়ার আগে (individual row-এর উপর), **HAVING** কাজ করে গ্রুপিং হওয়ার পরে (aggregated group-এর উপর)। তাই **COUNT(*)**, **SUM()** ইত্যাদি aggregate function দিয়ে ফিল্টার করতে হলে সবসময় **HAVING** লাগবে, **WHERE**-এ লেখা যাবে না।

এই প্রজেক্টে বাস্তব উদাহরণ

"প্রকাশিত এবং ব্রেকিং নিউজ, সর্বশেষ প্রথমে, ২০টা দেখাও" — এটা লিখলে:

কুয়েরি অংশ	কোড
FROM	articles
WHERE	status = 'published' AND is_breaking = 1
ORDER BY	published_at DESC
LIMIT	20

- প্রশ্ন: "যেসব ক্যাটাগরিতে ১০টার বেশি আর্টিকেল আছে, সেগুলো দেখাও" — এখানে **WHERE** লাগবে নাকি **HAVING**?
- প্রশ্ন: **LIMIT 20 OFFSET 40** বললে এটা কোন পেজের ডেটা দেখাবে (২০-প্রতি-পেজ ধরলে)?

মডিউল ২

JOIN — একাধিক টেবিল সংযুক্ত করা

প্রতিটা JOIN টাইপ কখন ব্যবহার করবে

কেন JOIN দরকার

Normalized ডেটাবেজে ডেটা একাধিক টেবিলে ছড়ানো থাকে (যেমন article-এর author আলাদা users টেবিলে) — একটা বোঝার মতো রেজাল্ট পেতে টেবিলগুলো জোড়া লাগাতে হয়। এটাই JOIN-এর কাজ।

INNER JOIN — শুধু মিল থাকা সারি

দুই টেবিলেই যে সারিগুলোর মিল আছে শুধু সেগুলো রিটার্ন করে। কোনো এক পাশে না থাকলে সেই সারি বাদ পড়ে যায়।

কোড	ব্যাখ্যা
SELECT a.headline, u.first_name FROM articles a INNER JOIN users u ON a.created_by = u.id	যেসব আর্টিকেলের creator আছে (এবং সেই user টেবিলে বিদ্যমান) শুধু সেগুলো দেখাবে

- এই প্রজেক্টে: article_author pivot দিয়ে article ↔ author জোড়া দেওয়া (INNER JOIN স্বাভাবিক পছন্দ, কারণ author ছাড়া article অর্থহীন)।

LEFT JOIN — বাম টেবিলের সব সারি, ডান পাশে মিল না থাকলেও

বাম টেবিলের (FROM-এর প্রথম টেবিল) সব সারি থাকবে, ডান পাশে মিল না পেলে NULL বসে যাবে। "থাকতে পারে বা নাও থাকতে পারে" সম্পর্কের জন্য এটাই সঠিক পছন্দ।

কোড	ব্যাখ্যা
SELECT a.headline, c.comment_count FROM articles a LEFT JOIN (SELECT article_id, COUNT(*) comment_count FROM comments GROUP BY article_id) c ON a.id = c.article_id	যে আর্টিকলে একটাও কমেন্ট নেই, তারও রেজাল্ট থাকবে (comment_count = NULL/0)। INNER JOIN দিলে কমেন্ট-শূন্য আর্টিকেল বাদ পড়ে যেত — যেটা ভুল ফলাফল দিত।

- প্রশ্ন: "প্রতিটা ক্যাটাগরি ও তার আর্টিকেল সংখ্যা দেখাও, যেসব ক্যাটাগরিতে একটাও আর্টিকেল নেই সেগুলোও যেন দেখায়" — INNER নাকি LEFT JOIN?

Self-JOIN — একই টেবিলকে নিজের সাথে জোড়া

যখন একই টেবিলের এক সারির সাথে আরেক সারির সম্পর্ক থাকে (হায়ারার্কি/ট্রি স্ট্রাকচার)।

কোড	ব্যাখ্যা
SELECT child.name AS subcategory, parent.name AS main_category FROM categories child LEFT JOIN categories parent ON child.parent_id = parent.id	categories.parent_id নিজেই categories.id-কে নির্দেশ করে — একই টেবিলকে দুইবার আলাদা alias (child, parent) দিয়ে জোড়া লাগানো হয়েছে

- এই প্রজেক্টে self-JOIN লাগে: categories (parent_id), comments (parent_id — থ্রেডেড রিপ্লাই), classified_categories, menu_items, editorial_desks (parent_desk_id)।

তবে গভীর হায়ারার্কি (৩+ লেভেল) query করতে self-JOIN বারবার লিখতে হয় (প্রতি লেভেলের জন্য এক জোড়া JOIN) — এই কারণেই categories/comments টেবিলে materialized_path কলাম রাখা হয়েছে, যাতে এক WHERE ... LIKE দিয়েই পুরো সারি ট্রি পাওয়া যায়, বারবার self-JOIN লিখতে না হয়।

CROSS JOIN — খুব কম ব্যবহার হয়

দুই টেবিলের প্রতিটা সারির সাথে প্রতিটা সারির সমন্বয় (Cartesian product) — সাধারণত ভুলবশত হয়ে যায় যখন JOIN কন্ডিশন (ON) দিতে ভুলে যাওয়া হয়। ইচ্ছাকৃত ব্যবহার খুবই কম (যেমন সব সম্ভাব্য size×color কম্বিনেশন জেনারেট করা)।

সতর্কতা: *ON* ক্লজ ছাড়া *JOIN* লিখলে *MySQL* এরর দেয় না, কিন্তু ফলাফল হবে বিশাল ভুল ডেটাসেট ($n \times m$ সারি) — এটা একটা সাধারণ বিগিনার ভুল।

Aggregation — গণনা, যোগফল, গড়

COUNT, SUM, AVG, GROUP BY, HAVING

মূল Aggregate Functions

Function	কাজ	উদাহরণ
COUNT()	সারি গণনা	একটা ক্যাটাগরিতে কতগুলো আর্টিকেল আছে
SUM()	যোগফল	একজন ফ্রিল্যান্সারের মোট প্রাপ্য টাকা (invoice_items.amount)
AVG()	গড়	গড় dwell_time_seconds (behavioral_metrics)
MAX() / MIN()	সর্বোচ্চ/সর্বনিম্ন	সবচেয়ে বেশি ভিউ পাওয়া আর্টিকেল

GROUP BY — কীভাবে গ্রুপ তৈরি হয়

SELECT-এ যে কলাম aggregate function-এর বাইরে থাকে, সেটাই GROUP BY-তে থাকতে হয় — এটাই নিয়ম। "প্রতিটা ক্যাটাগরির জন্য একটা রেজাল্ট" মানে category_id দিয়ে গ্রুপ করা।

কোড	ব্যাখ্যা
SELECT primary_category_id, COUNT(*) AS total FROM articles WHERE status = 'published' GROUP BY primary_category_id ORDER BY total DESC	প্রতিটা ক্যাটাগরিতে কতগুলো পাবলিশড আর্টিকেল আছে, বেশি থেকে কম সাজানো

HAVING — গ্রুপ ফিল্টার করা

WHERE কাজ করে গ্রুপিং হওয়ার আগে (raw row-এর উপর), HAVING কাজ করে গ্রুপিং হওয়ার পরে (aggregated ফলাফলের উপর)।

কোড	ব্যাখ্যা
SELECT wire_source_id, COUNT(*) AS pending_count FROM wire_ingestions WHERE status = 'pending' GROUP BY wire_source_id HAVING COUNT(*) > 50	যেসব wire source-এ ৫০-এর বেশি পেন্ডিং আইটেম জমে আছে (রিভিউ ব্যাকলগ চিহ্নিত করার জন্য) — এখানে HAVING লাগবে কারণ COUNT(*) একটা aggregate ফলাফল

- প্রশ্ন: "যেসব ইউজার এ মাসে ৫টার বেশি আর্টিকেল পড়েছে" — এই কুয়েরিতে WHERE আর HAVING দুটোই লাগবে কেন (হিন্ট: WHERE দিয়ে মাস ফিল্টার, HAVING দিয়ে কাউন্ট ফিল্টার)?

এই প্রজেক্টের বাস্তব ব্যবহার — Paywall Metering

একজন ইউজার এই মাসে কয়টা আর্টিকেল পড়েছে গণনা করা (read_histories টেবিল থেকে, মেটার্ড পেওয়াল চেক করতে):

কোড	ব্যাখ্যা
SELECT COUNT(DISTINCT article_id) AS articles_read FROM read_histories WHERE user_id = ? AND read_at >= DATE_FORMAT(NOW(), '%Y-%m-01')	DISTINCT জরুরি — একই আর্টিকেল একাধিকবার পড়লে যেন একবারই গণনা হয়। বাস্তবে এই কুয়েরি partition pruning-এর সুবিধা পায় কারণ read_at পার্টিশন কী।

Subquery ও CTE

কুয়েরির ভেতরে কুয়েরি — জটিল লজিক ধাপে ধাপে ভাঙা

Subquery — তিন ধরনের ব্যবহার

১. WHERE-এর ভেতরে (Scalar/IN subquery)

কোড	ব্যাখ্যা
SELECT * FROM articles WHERE primary_category_id IN (SELECT id FROM categories WHERE materialized_path LIKE '1/1/%')	প্রথমে ভেতরের কুয়েরি ক্যাটাগরি-১-এর সব সাব-ক্যাটাগরির ID বের করে, তারপর বাইরের কুয়েরি সেই ID-গুলোর যেকোনো একটাতে থাকা আর্টিকেল খোঁজে

২. FROM-এর ভেতরে (Derived table)

কোড	ব্যাখ্যা
SELECT cat.name, stats.total FROM categories cat JOIN (SELECT primary_category_id, COUNT(*) AS total FROM articles GROUP BY primary_category_id) stats ON cat.id = stats.primary_category_id	ভেতরের কুয়েরি আগে একটা "ভার্চুয়াল টেবিল" (stats) বানায়, তারপর সেটাকে categories-এর সাথে JOIN করা হয় — জটিল aggregation-কে আলাদা করে সহজপাঠ্য রাখার কৌশল

CTE (Common Table Expression) — WITH ক্লজ

CTE হলো subquery-রই আরেকটা ফরম্যাট, কিন্তু অনেক বেশি readable — জটিল কুয়েরিকে নামসহ ধাপে ধাপে ভাগ করা যায়, আর একই CTE একাধিকবার রেফারেন্স করা যায় (subquery-তে বারবার লিখতে হতো)।

কোড	ব্যাখ্যা
WITH monthly_reads AS (SELECT user_id, COUNT(DISTINCT article_id) AS cnt FROM read_histories WHERE read_at >= DATE_FORMAT(NOW(), '%Y-%m-01') GROUP BY user_id) SELECT u.email, mr.cnt FROM users u JOIN monthly_reads mr ON u.id = mr.user_id WHERE mr.cnt >= 5	প্রথমে monthly_reads নামে একটা টেম্পোরারি ফলাফল তৈরি হয়, তারপর সেটাকে সাধারণ টেবিলের মতোই ব্যবহার করা হয় — কোড অনেক পরিষ্কার হয়

MySQL 8+ (এই প্রজেক্টে যেটা ব্যবহার হচ্ছে) CTE সাপোর্ট করে, এমনকি Recursive CTE-ও (নিজেকে রেফারেন্স করা CTE, ট্রি-স্ট্রাকচার হাঁটার জন্য)। তবে categories/comments-এর মতো টেবিলে recursive CTE-এর বদলে materialized_path বেছে নেওয়া হয়েছে — কারণ MySQL-এ recursive CTE বড় ডেটাসেটে ধীরগতির (মডিউল-৫ দেখো)।

- প্রশ্ন: Recursive CTE কী, materialized_path কেন সেটার চেয়ে দ্রুত এই প্রজেক্টের স্কেলে?

Window Functions

গ্রুপ না করেও প্রতিটি সারিতে aggregate/rank তথ্য দেখানো

GROUP BY আর Window Function-এর পার্থক্য

GROUP BY একাধিক সারিকে একটা সারিতে সংকুচিত করে (detail হারিয়ে যায়)। Window Function প্রতিটি আলাদা সারি অক্ষত রেখে তার পাশে একটা "গ্রুপ-লেভেল" তথ্য (rank, running total, ইত্যাদি) যোগ করে দেয় — OVER() ক্লজ দিয়ে চেনা যায়।

ROW_NUMBER() / RANK() — প্রতিটি গ্রুপে টপ-N বের করা

কোড	ব্যাখ্যা
<pre>SELECT * FROM (SELECT id, headline, primary_category_id, view_count, ROW_NUMBER() OVER (PARTITION BY primary_category_id ORDER BY view_count DESC) AS rn FROM articles WHERE status = 'published') ranked WHERE rn <= 5</pre>	প্রতিটি ক্যাটাগরির জন্য আলাদাভাবে টপ-৫ সবচেয়ে বেশি দেখা আর্টিকেল বের করে — PARTITION BY এখানে GROUP BY-এর মতো কাজ করে কিন্তু সারিগুলো নার্জ হয় না

- এই প্রজেক্টে সরাসরি ব্যবহার: "প্রতি ক্যাটাগরির টপ ট্রেন্ডিং আর্টিকেল", "প্রতি ডেস্কের সাম্প্রতিক ৩টা অ্যাসাইনমেন্ট" — যেকোনো "প্রতি-গ্রুপে-টপ-N" প্রশ্নের জন্য।

LAG() / LEAD() — আগের/পরের সারির সাথে তুলনা

কোড	ব্যাখ্যা
<pre>SELECT snapshot_date, total_views, total_views - LAG(total_views) OVER (PARTITION BY article_id ORDER BY snapshot_date) AS view_change FROM content_performance_snapshots WHERE article_id = ?</pre>	প্রতিদিনের ভিউয়ের সাথে আগের দিনের তুলনা করে বৃদ্ধি/হ্রাস বের করা — trending:velocity গণনার পেছনের ধারণা এটাই

SUM() OVER() — Running Total

কোড	ব্যাখ্যা
<pre>SELECT invoice_id, amount, SUM(amount) OVER (PARTITION BY invoice_id ORDER BY id) AS running_total FROM invoice_items</pre>	প্রতিটা লাইন-আইটেমের সাথে ততক্ষণ পর্যন্ত মোট যোগফল দেখায় — ইনভয়েসের সাব-টোটাল ট্র্যাক করার জন্য উপকারী

Window function সবসময় aggregation-এর চেয়ে "বেশি খরচ করে না" — বরং GROUP BY দিয়ে একই কাজ করতে গেলে সাধারণত সাক-কুয়েরি বা self-JOIN লাগত, যেটা ধীরগতির। MySQL 8+-এই এই ফিচার এসেছে (৫.৭-এ ছিল না)।

- প্রশ্ন: RANK() আর ROW_NUMBER()-এর পার্থক্য কী (হিট টাই হলে কী হয়)?

Index ও Query Planner

কুয়েরি দ্রুত করার আসল রহস্য

Index আসলে কী

একটা বইয়ের ইনডেক্স যেমন প্রতিটা পাতা না উল্টে সরাসরি বিষয়ে পৌঁছে দেয়, ডেটাবেজ ইনডেক্সও তেমন — একটা আলাদা, সাজানো ডেটা স্ট্রাকচার (সাধারণত B-Tree) যা কলামের ভ্যালু থেকে সরাসরি রো-এর অবস্থানে নিয়ে যায়। ইনডেক্স ছাড়া MySQL-কে "full table scan" করতে হয় — প্রতিটা রো একে একে চেক করা, যা বড় টেবিলে ধ্বংসাত্মকভাবে ধীর।

Composite Index — কলামের অর্ডার গুরুত্বপূর্ণ

একাধিক কলামের উপর একসাথে ইনডেক্স বানালে কলামের ক্রম কুয়েরির পারফরম্যান্সে সরাসরি প্রভাব ফেলে — এটা "Leftmost Prefix Rule" নামে পরিচিত।

ইনডেক্স	যে কুয়েরিতে কাজ করবে	যে কুয়েরিতে কাজ করবে না
INDEX(status, published_at)	WHERE status=? — কাজ করবে WHERE status=? AND published_at>? — সম্পূর্ণ কাজ করবে	WHERE published_at>? (status ছাড়া) — কাজ করবে না, কারণ leftmost কলাম (status) কুয়েরিতে নেই

এই কারণেই `articles` টেবিলে `idx_published_listing` ইনডেক্স (`status, published_at DESC`) এই অর্ডারে বানানো হয়েছে — কারণ প্রতিটা লিস্টিং কুয়েরি সবসময় `status='published'` দিয়ে ফিল্টার শুরু করে, তারপর `published_at` দিয়ে সাজায়। উল্টো অর্ডারে বানালে ইনডেক্স কম কার্যকর হতো।

EXPLAIN — কুয়েরি আসলে কী করছে দেখা

কোড	ব্যাখ্যা
EXPLAIN SELECT * FROM articles WHERE status = 'published' ORDER BY published_at DESC LIMIT 20;	এই কমান্ড রান না করে শুধু MySQL-এর প্ল্যান দেখায় — কোন ইনডেক্স ব্যবহার হচ্ছে (key কলাম), কতগুলো রো স্ক্যান করতে হবে (rows কলাম), টাইপ কেমন (type কলাম — ALL মানে খারাপ full scan, ref/range মানে ভালো)

- `type=ALL` দেখলে বুঝবে ইনডেক্স ব্যবহার হচ্ছে না — বা তো ইনডেক্স নেই, না হয় কুয়েরি এমনভাবে লেখা যে ইনডেক্স ব্যবহার করা যাচ্ছে না (যেমন WHERE কলামের উপর ফাংশন প্রয়োগ করা: `WHERE YEAR(created_at)=2026` — এটা ইনডেক্স ব্যবহার করতে পারবে না)।
- প্রশ্ন: `EXPLAIN PARTITIONS` কী দেখায়, আর কেন এটা `partitioned` টেবিলে চেক করা জরুরি (মডিউল-১০ দেখো)?

Covering Index — সবচেয়ে দ্রুত সম্ভব কুয়েরি

যদি কুয়েরির প্রয়োজনীয় সব কলাম ইনডেক্সেই থাকে, MySQL মূল টেবিলে ফিরে না গিয়েই ইনডেক্স থেকে সরাসরি উত্তর দিতে পারে — একে "covering index" বলে, এটা সবচেয়ে দ্রুত।

N+1 সমস্যা ও Eager Loading

সবচেয়ে সাধারণ পারফরম্যান্স ভুল, যা টেস্টে ধরা পড়ে না

N+1 সমস্যা কী

মনে করো ২০টা আর্টিকেল দেখাতে হবে, প্রতিটার সাথে **author**-এর নাম। ভুল পদ্ধতি: প্রথমে ২০টা আর্টিকেল আনতে ১টা কুয়েরি, তারপর প্রতিটা আর্টিকেলের জন্য আলাদা করে **author** খুঁজতে আরও ২০টা কুয়েরি — মোট $1+20 = 21$ টা কুয়েরি। এটাই **N+1** সমস্যা।

ভুল পদ্ধতি (N+1)	সঠিক পদ্ধতি (Eager Loading)
<pre>foreach (articles as article) { \$author = User::find(article.created_by); } // ১ + ২০ = ২১টা কুয়েরি</pre>	<pre>Article::with('author')->get(); // মাত্র ২টা কুয়েরি: articles আনো, // তারপর WHERE id IN (...) দিয়ে // এক কুয়েরিতেই সব author আনো</pre>

এই সমস্যা ডেভেলপমেন্টের সময় ধরা পড়ে না কারণ কয়েকটা টেস্ট রো নিয়ে পার্থক্য বোঝা যায় না — শুধু প্রোডাকশনে বড় ডেটাসেটে ধীরগতি প্রকাশ পায়। এই কারণেই **nested response** থাকলে **selectinload()/with()** ব্যবহার করা এই প্রজেক্টে ঐচ্ছিক অপ্টিমাইজেশন না, বাধ্যতামূলক নিয়ম হিসেবে রাখা হয়েছে (মডিউল-১০-এর নোট দেখো)।

SQL-এর দিক থেকে এটা কী সমাধান করে

Eager loading আসলে ভেতরে ভেতরে দুটো কুয়েরি চালায়:

ধাপ	কুয়েরি
১	SELECT * FROM articles WHERE status='published' LIMIT 20
২	SELECT * FROM users WHERE id IN (৫, ১২, ৭, ...) — ধাপ-১ থেকে পাওয়া সব created_by ID একসাথে

এরপর অ্যাপ্লিকেশন কোডে (মেমরিতে) দুটো ফলাফল মিলিয়ে নেওয়া হয় — এটাকে "batch loading" বলা যায়। ২১টা ছোট কুয়েরির বদলে ২টা কুয়েরি, প্রতিটা **network round-trip**-এর খরচ বাঁচায়।

- প্রশ্ন: **nested relationship** (`article` → `author` → `author-institutional_account`) তিন লেভেল গভীর হলে **eager loading** কীভাবে কাজ করবে (হিন্ট: `with('author.institutionalAccount')`)?

মডিউল ৮

Full-text Search ও JSON কুয়েরি

সাধারণ WHERE = দিয়ে যা সম্ভব না

Full-text Search — LIKE কেন যথেষ্ট না

পদ্ধতি	সমস্যা/সুবিধা
WHERE headline LIKE '%নির্বাচন%'	কাজ করে কিন্তু ইনডেক্স ব্যবহার করতে পারে না (leading wildcard % দিয়ে শুরু) — বড় টেবিলে ধীরগতির, প্রতিটা রো স্ক্যান করতে হয়
WHERE MATCH(headline, sub_headline) AGAINST('নির্বাচন' IN NATURAL LANGUAGE MODE)	FULLTEXT ইনডেক্স ব্যবহার করে, শব্দ-ভিত্তিক relevance স্কোর দেয়, দ্রুত — কিন্তু শুধু FULLTEXT ইনডেক্স করা কলামেই কাজ করে

এই প্রজেক্টে আসল সার্চ Meilisearch করে (typo-tolerant, বাংলা সাপোর্ট ভালো), কিন্তু articles টেবিলে FULLTEXT(headline, sub_headline) ইনডেক্সও রাখা হয়েছে fallback হিসেবে — Meilisearch ডাউন থাকলেও বেসিক সার্চ কাজ করবে। "single point of failure" এড়ানোর একটা প্র্যাকটিক্যাল উদাহরণ এটা।

JSON কলাম কুয়েরি করা

এই প্রজেক্টে অনেক কলাম JSON (features, target_categories, geo_restriction, embedding_vector) — MySQL 5.7+ এ JSON কলামের ভেতরে সরাসরি কুয়েরি করা যায়।

কোড	ব্যাখ্যা
SELECT * FROM plans WHERE JSON_CONTAINS(features, "podcast_access")	যেসব প্লানে podcast_access ফিচার আছে — JSON array-এর ভেতরে একটা ভ্যালু আছে কিনা চেক করে
SELECT * FROM articles WHERE JSON_CONTAINS(geo_restriction, "BD")	যেসব আর্টিকেল বাংলাদেশে সীমাবদ্ধ
SELECT title, JSON_EXTRACT(condition_logic, '\$.limit') AS limit_val FROM paywall_rules	JSON অবজেক্টের নির্দিষ্ট key-এর ভ্যালু বের করা

সতর্কতা — JSON কলামে সাধারণ ইনডেক্স কাজ করে না, দরকার হলে "generated column" বানিয়ে তার উপর ইনডেক্স করতে হয় (JSON থেকে একটা virtual কলাম বের করে ইনডেক্স করা)। এই কারণেই যেসব কলামে ঘনঘন filter করা হবে (যেমন status), সেগুলো JSON-এর ভেতরে না রেখে আলাদা সাধারণ কলাম হিসেবে রাখা হয়েছে — JSON শুধু flexible/কম ফিল্টার-হওয়া ডেটার জন্য।

- প্রশ্ন: কেন paywall_rules.condition_logic JSON রাখা হলো কিন্তু articles.status ENUM/VARCHAR রাখা হলো, দুটোই তো flexible data — পার্থক্যটা কোথায়?

Partitioning ও Partition Pruning

কোটি কোটি রো-এর টেবিলে কীভাবে কুয়েরি দ্রুত রাখা যায়

পার্টিশন কী করে ভেতরে ভেতরে

behavioral_metrics টেবিল `RANGE BY (YEAR(recorded_at)*100+MONTH(recorded_at))` দিয়ে পার্টিশন করা — অর্থাৎ প্রতিটা মাসের ডেটা আলাদা "সাব-টেবিলে" (পার্টিশনে) থাকে, যদিও উপর থেকে একটাই টেবিল মনে হয়।

কোড	ব্যাখ্যা
<code>SELECT COUNT(*) FROM behavioral_metrics WHERE recorded_at >= '2026-08-01' AND recorded_at < '2026-09-01'</code>	MySQL query planner বুঝে যায় এই কুয়েরির উত্তর শুধু আগস্ট-২০২৬ পার্টিশনে আছে — বাকি ২৩+ মাসের পার্টিশন সম্পূর্ণ স্কিপ করে দেয়, একবারও ছুঁয়ে দেখে না। এটাই "partition pruning"।

Partition pruning কাজ করে শুধুমাত্র যখন WHERE ক্লজে পার্টিশন-কী কলাম (এখানে recorded_at) থাকে। যদি কুয়েরি লেখা হয় WHERE article_id=? (পার্টিশন-কী ছাড়া), তাহলে MySQL-কে সব পার্টিশন চেক করতে হবে — pruning কাজ করবে না, পারফরম্যান্স স্বাভাবিক non-partitioned টেবিলের মতোই খারাপ হবে।

EXPLAIN PARTITIONS দিয়ে যাচাই করা

কোড	ব্যাখ্যা
<code>EXPLAIN PARTITIONS SELECT * FROM read_histories WHERE user_id = 5 AND read_at >= '2026-08-01';</code>	partitions কলামে যদি শুধু "p202608" দেখায় — pruning কাজ করছে (ভালো)। যদি সব পার্টিশনের নাম দেখায় (p202401, p202402...) — pruning কাজ করছে না (খারাপ, fix করতে হবে)

কেন **Partitioned** টেবিলে **Foreign Key** কাজ করে না

MySQL partitioned টেবিলে FK constraint সাপোর্ট করে না — কারণ একটা FK চেক করতে হলে referenced টেবিলের নির্দিষ্ট পার্টিশনে দ্রুত পৌঁছাতে হয়, কিন্তু FK-এর সাথে partition-key সবসময় সম্পর্কিত না থাকায় এই গ্যারান্টি দেওয়া কঠিন। তাই এই প্রজেক্টে read_histories, behavioral_metrics, activity_logs-এ FK নেই — referential integrity Eloquent relationship (অ্যান্নিকেশন লেয়ার) দিয়ে সামলানো হয়।

- প্রশ্ন: partition pruning কাজ না করলে কী সমস্যা হবে ১০ কোটি রো-এর টেবিলে?

Pagination — Offset vs Cursor

পেজিনেশনের দুই পদ্ধতি ও কেন একটা আরেকটার চেয়ে ভালো

Offset Pagination — সহজ কিন্তু স্কেলে সমস্যা

কোড	ব্যাখ্যা
SELECT * FROM articles WHERE status='published' ORDER BY published_at DESC LIMIT 20 OFFSET 10000	পেজ ৫০১ (২০-প্রতি-পেজ) দেখাতে MySQL-কে প্রথমে ১০,০২০টা রো স্ক্যান করতে হয়, তারপর প্রথম ১০,০০০টা ফেলে দিয়ে পরের ২০টা রিটার্ন করে — OFFSET যত বড় হয় কুয়েরি তত ধীর হয়

Cursor (Keyset) Pagination — স্কেলে দ্রুত

কোড	ব্যাখ্যা
SELECT * FROM articles WHERE status='published' AND published_at < ? -- আগের পেজের শেষ published_at ORDER BY published_at DESC LIMIT 20	OFFSET-এর বদলে "আগের পেজের শেষ আইটেমের পরে থেকে শুরু করো" — ইনডেক্স ব্যবহার করে সরাসরি সেই বিন্দুতে জাম্প করে, স্ক্যান করার দরকার নেই। পেজ ৫ হোক বা পেজ ৫০০, গতি প্রায় একই থাকে

Trade-off: cursor pagination-এ "সরাসরি পেজ ৫০-এ যাও" বলা কঠিন (sequential navigation লাগে), কিন্তু infinite-scroll ফিড (হোমপেজ, নিউজফিড) এর জন্য এটাই আদর্শ — ঠিক যেভাবে Facebook/Twitter ফিড কাজ করে। এই প্রজেক্টে হোমপেজ/ক্যাটাগরি পেজ ফিডে cursor pagination আর অ্যাডমিন প্যানেলের মতো জায়গায় (যেখানে নির্দিষ্ট পেজ নাম্বারে যাওয়া দরকার) offset pagination — দুটো ভিন্ন ব্যবহারের জন্য দুটো ভিন্ন কৌশল।

- প্রশ্ন: cursor pagination-এ "পেজ ৭-এ সরাসরি যাও" কেন কঠিন?

Transaction ও Locking

একসাথে একাধিক পরিবর্তন নিরাপদ রাখা, রেস কন্ডিশন এড়ানো

Transaction — সব হবে, নয়তো কিছুই না (ACID-এর "A")

কোড	ব্যাখ্যা
START TRANSACTION; UPDATE subscriptions SET status='active' WHERE id=?; INSERT INTO subscription_events (...) VALUES (...); INSERT INTO transactions (...) VALUES (...); COMMIT;	তিনটা অপারেশন একসাথে সফল হবে অথবা কোনোটাই হবে না — মাঝপথে সার্ভার ক্র্যাশ করলে ROLLBACK হয়ে যায়, অর্ধেক-সম্পন্ন অবস্থায় ডেটা আটকে থাকে না

আর্টিকেল পাবলিশ করার সময়ও (মডিউল-৬) transaction ব্যবহার হয়: status আপডেট + published_at সেট — এই দুটো একসাথে atomic হতে হবে, নাহলে "published কিন্তু published_at খালি" এমন অসম্ভব অবস্থা তৈরি হতে পারে।

Race Condition — কেন লক দরকার

দুইজন এডিটর একই সময়ে একই আর্টিকেল সেভ করলে কী হবে? দুজনেই পুরনো ভার্সন পড়ে, নিজের মতো এডিট করে সেভ করে — যে শেষে সেভ করে সে আরেকজনের পরিবর্তন মুছে ফেলে (silent data loss)। একেই race condition বলে।

Pessimistic Locking — SELECT ... FOR UPDATE

কোড	ব্যাখ্যা
START TRANSACTION; SELECT * FROM ad_campaigns WHERE id=? FOR UPDATE; -- spent_amount চেক করে বাজেটের মধ্যে থাকলে UPDATE ad_campaigns SET spent_amount=spent_amount+? WHERE id=?; COMMIT;	FOR UPDATE দিয়ে রো-টা লক করে দেওয়া হয় — অন্য কোনো ট্রানজ্যাকশন এই রো একসাথে পড়তে/বদলাতে পারবে না যতক্ষণ না প্রথম ট্রানজ্যাকশন COMMIT হয়। বাজেট ওভারস্পেন্ড এড়াতে জরুরি

Optimistic Locking — editorial_locks-এর প্যাটার্ন

এই প্রজেক্টে article এডিটিং-এর জন্য DB row-lock না নিয়ে বরং editorial_locks নামে আলাদা একটা "লক টেবিল" ব্যবহার করা হয়েছে (মডিউল-৬) — একজন এডিটর কাজ শুরু করলে সেখানে একটা রো তৈরি হয়, অন্য এডিটর চেষ্টা করলে অ্যাক্সিকেশন সেটা দেখে আটকে দেয়। এটা "advisory lock" প্যাটার্ন — DB-এর ট্রানজ্যাকশনাল লকের চেয়ে হালকা, কিন্তু দীর্ঘ সময় (মিনিট/ঘণ্টা) ধরে রাখার জন্য বেশি উপযোগী, কারণ FOR UPDATE-এর মতো ট্রানজ্যাকশন খোলা রাখতে হয় না।

- প্রশ্ন: FOR UPDATE লক আর editorial_locks টেবিল-ভিত্তিক লক — কোনটা কখন ব্যবহার করা উচিত (সংক্ষিপ্ত-দ্রুত অপারেশন vs দীর্ঘ-মানুষ-চালিত অপারেশন)?
- প্রশ্ন: FOR UPDATE লক নেওয়ার পর COMMIT/ROLLBACK না করে ট্রানজ্যাকশন খোলা রাখলে কী সমস্যা হতে পারে?

মডিউল ১২

এই প্রজেক্টের বাস্তব কুয়েরি প্যাটার্ন

ফিচার অনুযায়ী সাজানো — প্রতিটা এখন পর্যন্ত শেখা কনসেপ্টের বাস্তব প্রয়োগ

উপরের ১১টা কনসেপ্ট আলাদা আলাদা করে শেখার পর, এবার দেখা যাক প্রজেক্টের আসল ফিচারগুলোতে এগুলো কীভাবে একসাথে কাজ করে।

১. হোমপেজ ফিড (Cache-first + Cursor Pagination)

ধাপ	কী হয়
১	Redis চেক: article:slug অথবা homepage:feed:page1 key আছে কিনা
২	cache miss হলে SQL: SELECT * FROM articles WHERE status='published' ORDER BY published_at DESC LIMIT 20 (প্রথম পেজ, cursor ছাড়া)
৩	পরের পেজে cursor pagination: AND published_at < ? (আগের পেজের শেষ টাইমস্ট্যাম্প)
৪	ফলাফল Redis-এ cache (TTL 300s), তারপর রেসপন্স

এখানে Part ১ (basic SELECT/ORDER/LIMIT) + Part ১০ (cursor pagination) + Redis cache-first প্যাটার্ন — সব একসাথে কাজ করছে।

২. ক্যাটাগরি পেজ (সাব-ক্যাটাগরি সহ Materialized Path)

কোড	ব্যাখ্যা
SELECT a.* FROM articles a JOIN categories c ON a.primary_category_id = c.id WHERE c.materialized_path LIKE '/3/%' AND a.status = 'published' ORDER BY a.published_at DESC LIMIT 20	"খেলাধুলা" ক্যাটাগরি ও তার সব সাব-ক্যাটাগরির (ক্রিকেট, ফুটবল...) আর্টিকেল একসাথে দেখায় — materialized_path দিয়ে এক LIKE কুয়েরিতেই পুরো সাব-ট্রি কভার হয়

Part ২ (JOIN) + materialized path concept (self-referencing hierarchy-এর বিকল্প) একসাথে।

৩. ট্রেন্ডিং আর্টিকেল (Window Function + Aggregation)

কোড	ব্যাখ্যা
SELECT article_id, COUNT(*) AS views_last_hour, RANK() OVER (ORDER BY COUNT(*) DESC) AS rn FROM behavioral_metrics WHERE recorded_at >= NOW() - INTERVAL 1 HOUR GROUP BY article_id HAVING COUNT(*) > 10	গত ১ ঘণ্টায় সবচেয়ে বেশি পড়া আর্টিকেল, র‍্যাংক সহ — বাস্তবে এই গণনা Redis Sorted Set-এ রিয়েল-টাইমে হয় (trending:velocity:1h), কিন্তু ভিত্তি ধারণা এই SQL-ই

Part ৩ (GROUP BY + HAVING) + Part ৫ (Window Function RANK) + Part ৯ (partition pruning, কারণ behavioral_metrics পার্টিশনড) — তিনটা কনসেপ্ট একসাথে।

৪. পেওয়াল চেক (Subquery + Redis Cache)

ধাপ	কী হয়
১	Redis চেক: user:entitlements:{id} আছে কিনা (থাকলে সরাসরি সিদ্ধান্ত, DB-তে যেতে হয় না)
২	cache miss হলে: SELECT feature_code FROM user_entitlements WHERE user_id=? AND (expires_at IS NULL OR expires_at > NOW())

ধাপ	কী হয়
৩	যদি <code>unlimited_articles</code> না থাকে: <code>SELECT COUNT(DISTINCT article_id) FROM read_histories WHERE user_id=? AND read_at >=</code> এই-মাসের-শুরু (মেটার্ড লিমিট চেক)
৪	<code>paywall_rules</code> -এর <code>condition_logic</code> JSON থেকে <code>limit</code> বের করে তুলনা করা

এখানে *Part ৩ (COUNT/aggregation)*, *Part ৮ (JSON কুয়েরি)* এবং *Redis cache-first* প্যাটার্ন একসাথে কাজ করছে — এটাই দেখায় কেন *real-world* কুয়েরি কখনো একটা কনসেপ্টে সীমাবদ্ধ থাকে না।

৫. আর্টিকেল পাবলিশ (Transaction + Event-driven Cache Invalidation)

কোড	ব্যাখ্যা
START TRANSACTION; UPDATE articles SET status='published', published_at=NOW() WHERE id=? AND status='scheduled'; INSERT INTO article_revisions (...) VALUES (...,'publish'); COMMIT;	status ও published_at একসাথে বদলাতে হবে atomically। WHERE-এ status='scheduled' রাখা একটা optimistic check — যদি ইতিমধ্যে অন্য কেউ পাবলিশ করে ফেলে থাকে (status আর scheduled নেই), UPDATE কিছুই বদলাবে না (affected rows = 0), অ্যাপ্লিকেশন কোড সেটা বুঝে ইউজারকে জানাবে

| Part ১১ (Transaction) + একটা হালকা optimistic locking কৌশল (WHERE-এ পুরনো স্টেট চেক করা, আলাদা lock না নিয়েও)।

৬. সার্চ ফলব্যাক (Full-text + Relevance)

কোড	ব্যাখ্যা
SELECT id, headline, MATCH(headline, sub_headline) AGAINST(? IN NATURAL LANGUAGE MODE) AS relevance FROM articles WHERE MATCH(headline, sub_headline) AGAINST(? IN NATURAL LANGUAGE MODE) AND status = 'published' ORDER BY relevance DESC LIMIT 20	Meilisearch ডাউন থাকলে এই fallback কুয়েরি চলে — MATCH...AGAINST দুইবার লেখা হয়েছে: একবার SELECT-এ স্কোর দেখাতে, একবার WHERE-এ ফিল্টার করতে

৭. রেভিনিউ অ্যাট্রিবিউশন রিপোর্ট (Multi-table JOIN + Aggregation)

কোড	ব্যাখ্যা
SELECT a.headline, COUNT(ra.id) AS conversions, SUM(ra.attributed_revenue) AS total_revenue FROM revenue_attribution ra JOIN articles a ON ra.source_article_id = a.id WHERE ra.attribution_source = 'article_paywall' AND ra.created_at >= ? GROUP BY a.id ORDER BY total_revenue DESC LIMIT 10	"কোন আর্টিকেল সবচেয়ে বেশি সাবস্ক্রিপশন এনেছে" — এই একটা কুয়েরিতে JOIN, GROUP BY, দুই ধরনের aggregate function (COUNT, SUM), আর ORDER BY + LIMIT সবকিছু একসাথে ব্যবহৃত হচ্ছে

সিদ্ধান্ত-ইনডেক্স — কোন কনসেপ্ট কোথায় কাজে লাগে

কনসেপ্ট	এই প্রজেক্টে কোথায় লাগবে
JOIN (INNER/LEFT)	article-author, article-category, comment-thread, প্রায় প্রতিটা লিস্টিং পেজ
Self-JOIN / Materialized Path	categories, comments, classified_categories, menu_items — সব হায়ারার্কিক্যাল ডেটা
Aggregation (COUNT/SUM/GROUP BY)	paywall metering, revenue reports, ad analytics, ট্যাগ/ক্যাটাগরি কাউন্ট
HAVING	যেকোনো "গ্রুপের সংখ্যা/যোগফল দিয়ে ফিল্টার" — যেমন backlog thresholds, active user segments
Subquery / CTE	জটিল paywall rules evaluation, nested filtering, রিপোর্ট বিভিৎ
Window Function	ট্রেন্ডিং (per-category top-N), revenue ranking, running totals (invoice)
Composite Index + EXPLAIN	প্রতিটা "listing" কুয়েরি — homepage, category page, admin panel
N+1 / Eager Loading	nested API response — article+author+category+media সব একসাথে দেখানো যেকোনো এন্ডপয়েন্ট
Full-text Search	Meilisearch fallback, articles.headline/sub_headline
JSON Query	plans.features, paywall_rules.condition_logic, ad_campaigns targeting
Partition Pruning	behavioral_metrics, read_histories, activity_logs — সব অ্যানালিটিক্স রিপোর্ট
Cursor Pagination	হোমপেজ ফিড, ক্যাটাগরি পেজ, ইনফিনিট-স্ক্রল যেকোনো জায়গা
Offset Pagination	অ্যাডমিন প্যানেল, ফিল্টার পেজ-নাম্বার নেভিগেশন
Transaction	article publish, subscription lifecycle change, payment processing
Pessimistic Lock (FOR UPDATE)	ad_campaigns.spent_amount আপডেট, ticket_tiers.quantity_sold (টিকেট বিক্রি)
Advisory/Table Lock	editorial_locks — দীর্ঘ-সময়-ধরে-রাখা মানুষ-চালিত লক

চূড়ান্ত পরামর্শ — যখনই নতুন কোনো ফিচারের কুয়েরি লিখতে বসবে, নিজেকে প্রশ্ন করো: এটা কি একটা সাধারণ **SELECT (Part ১)**, নাকি একাধিক টেবিল লাগবে (**Part ২**), নাকি গণনা/গ্রুপিং লাগবে (**Part ৩**), নাকি "প্রতি-গ্রুপে" কিছু বের করতে হবে (**Part ৫ — Window Function**)? এই প্রশ্নগুলো ধাপে ধাপে করলে সঠিক কুয়েরি-প্যাটার্ন বেছে নেওয়া সহজ হয়ে যাবে।